

Tutorial 6: Multitasking

Nanyang Technological University

Q1 (a) State two distinct types of multitasking techniques commonly used in embedded OS based applications.

(b) How are these techniques implemented in a Linux based platform?

(c) Discuss the main differences between the executions of these techniques.

(d) Compares the features provided by each of these techniques, and how they can be effectively used in some typical real-world applications.

Q2 What is a process ID (PID), and why is it essential for process management in Linux?

Q3 Write a multi-process program, let the parent process to kill the child process via its process ID.

Q4 Describe the output that will be observed when the following programs are executed.

(a) Case 1: **child will copy the rest of parent code after fork(); Case1, hello world print 3 times**

```
1 int main()
2 {
3     printf("Hello World!\n");
4     fork();
5     printf("Hello World!\n");
6     return 0;
7 }
```

(b) Case 2: **See photo on 25 March 2026, 3 hello world, 2 from main, 1 from pid_if. Due to compiler optimisation, Pchild1child2 will not get created, as the first fork() in Pchild1 is already 0, so it will skip the 2nd fork();**

```
1 int main()
2 {
3     printf("Hello World!\n");
4     if( fork() && fork())
5     {
6         fork();
7         printf("Hello World!\n");
8     }
9     return 0;
10 }
```

Threads vs processes in linux, Threads are created from pthreads, the different threads have a shared memory, can interact with each other in the same process. Processes are essentially different programs, must communicate via pipes or other interprocess method.